Farrel Putra Imanuel The - 2802391883

Bagian A: Analisis Konseptual (Bobot: 20%)

1. Efek Regularisasi Batch Normalization (Slide 6–8)

Mekanisme (Slide 6–8). Pada Slide 7, μ dan σ^2 dihitung dari mini-batch berukuran m, bukan dari seluruh data. Karena isi batch diacak tiap iterasi, sebuah sampel dinormalisasi dengan statistik yang sedikit berbeda setiap kali muncul, sehingga aktivasinya berfluktuasi. Fluktuasi (noise) ini membuat model tidak bisa menghafal pola tepat per sampel, sehingga BN berperan sebagai regularizer ringan (Slide 6, 8).

Kapan "menghilangkan Dropout" tidak berlaku. Slide 8 hanya menyebut "*often*", bukan selalu:

- Batch sangat kecil: μ dan σ^2 dari m kecil terlalu noisy, sehingga pelatihan tidak stabil dan statistik tidak representatif.
- Overfitting tetap tinggi (data kecil, model besar, Slide 28): regularisasi BN yang "*mild*" tidak cukup, sehingga Dropout tetap diperlukan.
- Dropout ditempatkan sebelum BN: Dropout mengubah varians aktivasi saat training, tetapi tidak saat inferensi, sehingga statistik BN tidak cocok dan akurasi dapat turun.

2. Degradasi Jaringan Dalam dan Formulasi Residual (Slide 10–12)

Degradasi (Slide 10–12). Pada plain CNN, lapisan tambahan harus *mempelajari sendiri* fungsi identitas melalui tumpukan lapisan nonlinear, dan itu sulit dioptimasi. Slide 10 menyebut penyebabnya: gradient vanishing/explosion dan information loss antarlapisan. Akibatnya, jaringan lebih dalam bisa lebih buruk bahkan pada akurasi latih.

Aliran gradien. Slide 11: $y = F(x) + x$, sehingga $\partial L/\partial x = \partial L/\partial y \cdot (\partial F/\partial x + 1) = \partial L/\partial y \cdot \partial F/\partial x + \partial L/\partial y$. Suku "$+ \partial L/\partial y$" menyalurkan gradien langsung ke lapisan sebelumnya tanpa melewati bobot. Lapisan juga cukup belajar $F(x) \approx 0$ untuk meniru identitas.

Kaitan Sesi 02. Vanishing gradient terjadi karena perkalian turunan berantai yang kecil. Skip connection menambahkan jalur penjumlahan, sehingga gradien tetap mengalir ("*helps gradients flow*", Slide 11) dan jaringan sangat dalam stabil dilatih, misalnya ResNet-101 (Slide 12).

3. Feature Extraction vs. Fine-Tuning (Slide 14–16)

Perbandingan (Slide 14–16).

- Parameter: Feature Extraction (FE) hanya melatih classifier head baru (semua lapisan pretrained dibekukan). Fine-Tuning (FT) membuka sebagian lapisan, misalnya layer4 (Slide 18), hingga seluruhnya.

- Data: FE cocok untuk data terbatas (Slide 15). FT butuh data lebih banyak karena parameter yang dilatih lebih banyak, sehingga risiko overfitting naik (Slide 28).
- Komputasi/memori: FE ringan karena backbone tidak diperbarui. FT lebih berat karena gradien dihitung untuk lebih banyak lapisan.
- Risiko fitur: FE nyaris tanpa risiko karena bobot beku. FT dapat merusak fitur bawaan bila pembaruan terlalu agresif (Slide 28), sehingga memakai learning rate lebih rendah (Slide 16).

Kriteria: mulai dari FE (Slide 29), terutama bila data sedikit, domain mirip ImageNet, atau komputasi terbatas (Slide 15). Lanjut ke FT bertahap bila hasil FE kurang memadai dan data cukup.

4. Kaidah Penerapan Data Augmentation (Slide 20–22)

(a) Hanya pada train set. Augmentasi bertujuan menambah variasi data latih agar model lebih general (Slide 20). Validation/test berfungsi mengukur kinerja pada data asli. Jika diaugmentasi, hasil evaluasi menjadi acak dan tidak mencerminkan kondisi nyata, sehingga perbandingan antar-eksperimen tidak adil dan tidak bisa direproduksi. Karena itu, kode di Slide 22 (Flip, Rotation, ColorJitter) hanya dipakai pada transform data latih, sedangkan validation/test cukup resize dan ToTensor.

(b) Contoh tidak layak: Vertical Flip (Slide 21). Foto hewan peliharaan hampir selalu tegak. Gambar terbalik tidak pernah muncul pada data nyata, sehingga model belajar dari distribusi yang tidak relevan. Alternatif: color jitter ekstrem (hue). Warna dan pola bulu adalah ciri penentu ras, sehingga mengubahnya dapat membuat gambar menyerupai ras lain, sehingga label tidak lagi invarian terhadap transformasi. Brightness dan contrast ringan (Slide 21) tetap aman.

5. Mitigasi Catastrophic Forgetting (Slide 28–29)

Terjadinya (Slide 28). Bobot pretrained menyimpan fitur yang dipelajari dari dataset besar (Slide 14). Saat fine-tuning, classifier head baru masih acak sehingga gradien awalnya besar dan tidak informatif. Jika learning rate terlalu tinggi atau banyak lapisan dibuka sekaligus, gradien ini menggeser bobot backbone jauh dari nilai awalnya. Fitur bawaan tertimpa oleh pola data baru yang kecil, sehingga pengetahuan lama hilang.

Mitigasi:

1. Gradual unfreezing (Slide 16): latih head dahulu (feature extraction), lalu buka lapisan dalam secara bertahap. Alasannya, head sudah stabil sebelum backbone disentuh, dan lapisan awal yang berisi fitur umum diubah paling akhir atau paling sedikit.
2. Learning rate rendah + lr_scheduler (Slide 16, 29): langkah pembaruan kecil membuat bobot hanya bergeser sedikit dari hasil pretraining. Scheduler menurunkan learning rate seiring pelatihan, sehingga adaptasi tetap halus.